循序渐进，学习开发 xv6-riscv
第 12 章 多核处理

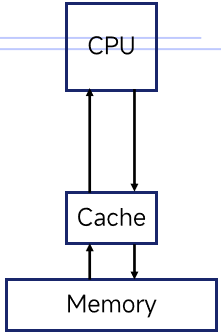
汪辰

目录

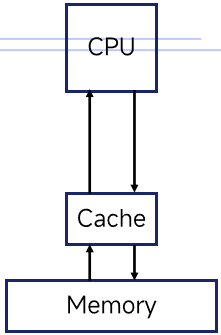
01

SMP 介绍

SMP 介绍

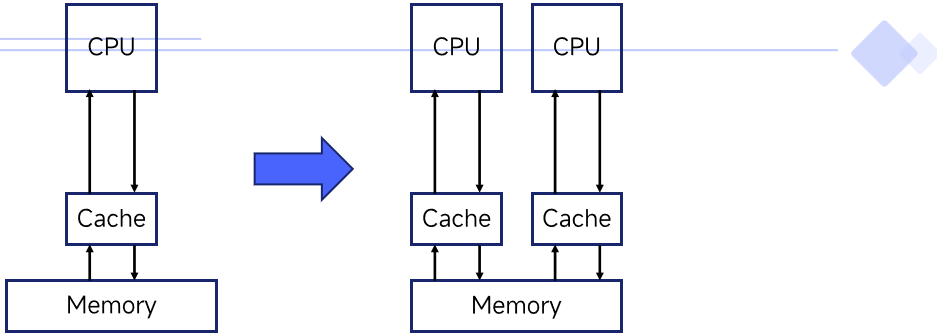


SMP 介绍

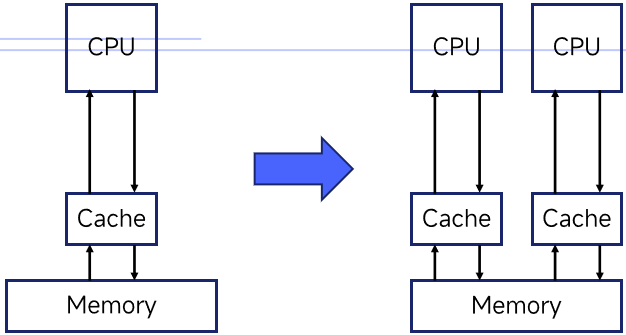


- 时间局部性原理
- 空间局部性原理
- 缓存的分级

SMP 介绍



SMP 介绍



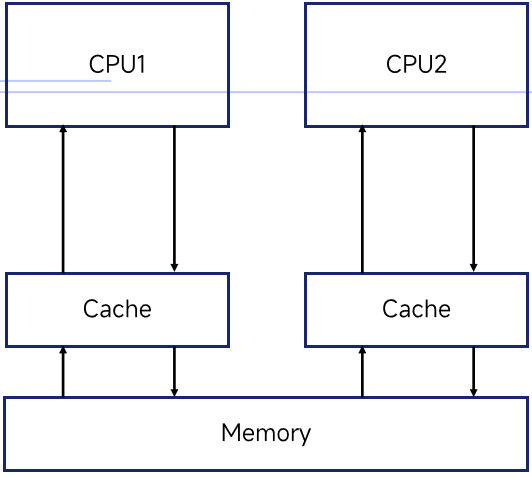
对称多处理
Symmetric Multi-Processing

02 SMP 涉及的典型问题

SMP 涉及的一些典型问题

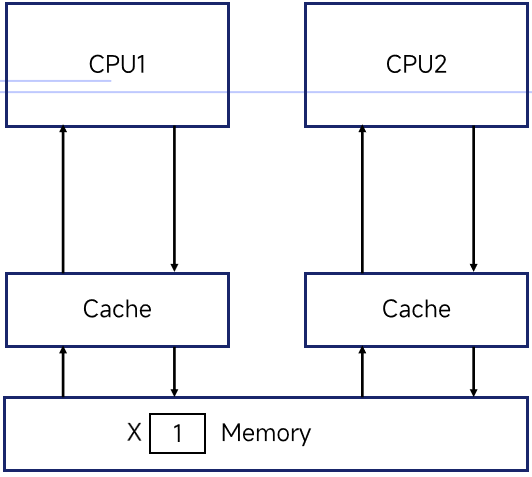
- 缓存一致性（Cache Coherence ）
- 内存一致性模型（Memory Consistency Model ）

缓存一致性问题



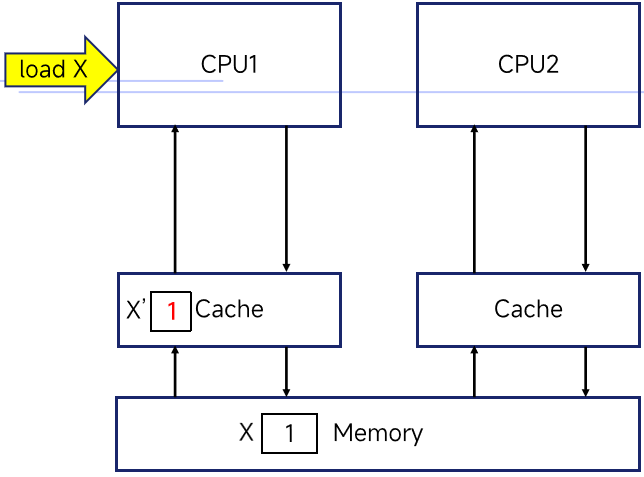
对称多处理
Symmetric Multi-Processing

缓存一致性问题



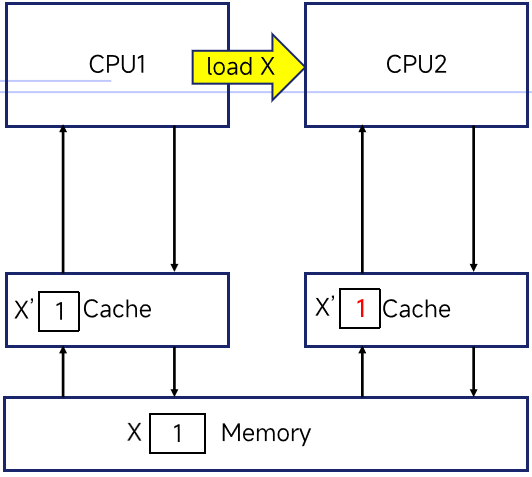
对称多处理
Symmetric Multi-Processing

缓存一致性问题 - CPU1 读取 X



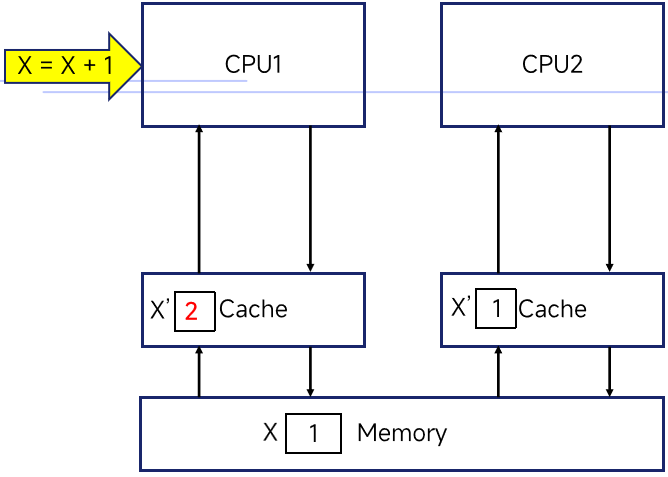
对称多处理
Symmetric Multi-Processing

缓存一致性问题 - CPU2 读取 X



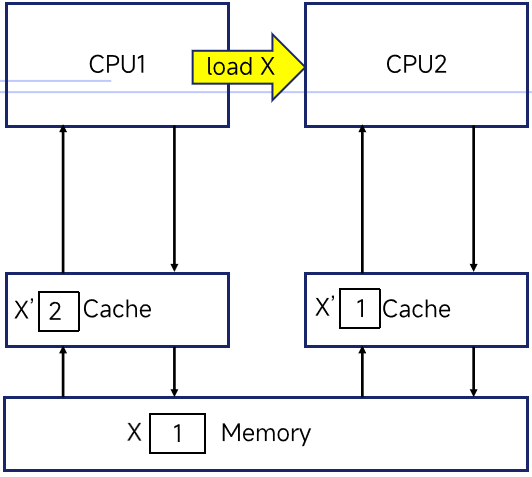
对称多处理
Symmetric Multi-Processing

缓存一致性问题 - CPU1 执行 X=X+1



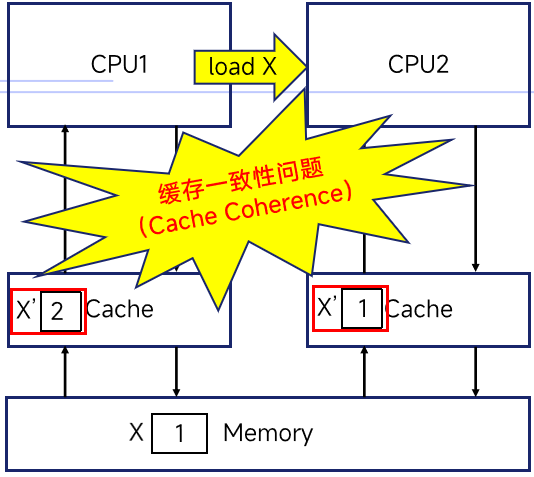
对称多处理
Symmetric Multi-Processing

缓存一致性问题 - CPU2 再次读取 X



对称多处理
Symmetric Multi-Processing

缓存一致性问题



对称多处理
Symmetric Multi-Processing

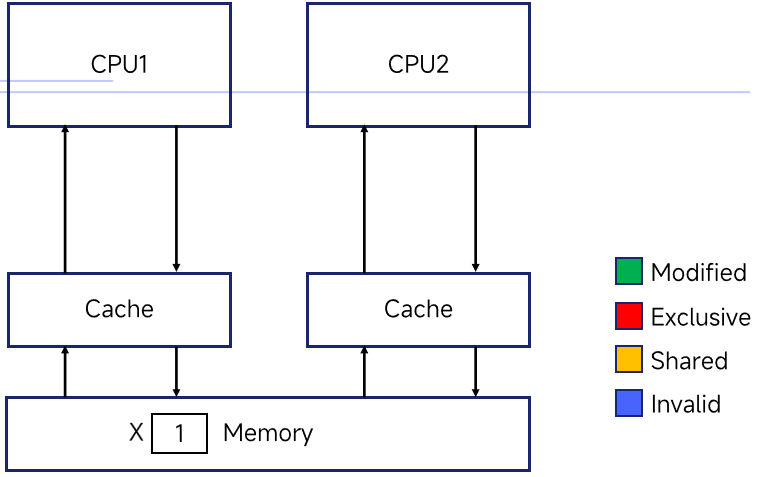
缓存一致性问题的解决方案 - MESI

硬件提供了缓存一致性问题的解决方案，其中最著名且应用最广泛的就是 MESI 协议。

- 在基于总线的系统中，通过总线嗅探技术（bus snooping），缓存控制器通过监听链接缓存和主存的总线来发现对内存的访问操作，并同时维护缓存行的状态。
- 缓存控制器基于以下这些状态来管理数据的传播和同步。

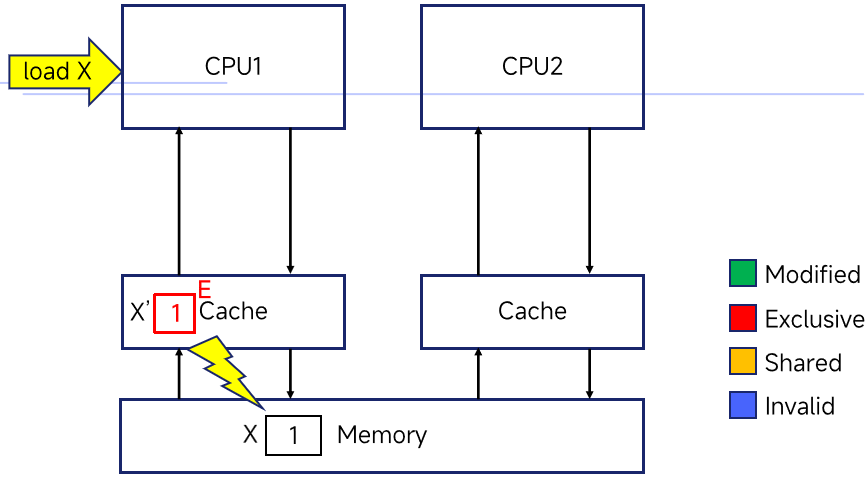
状态	含义	动作
M（Modified，已修改）	该缓存行中的数据是 **有效** 的，但 **已被修改**，与主内存中的数据 **不一致**。而且，这份数据只存在于当前核心的缓存中（在其他核心的缓存中不存在，或者已经失效）。	如果其他核心要读取这块内存地址，当前核心必须将数据 **写回主内存**（或者直接通过缓存间传输），并改变状态，保证其他核心能拿到最新数据。
E（Exclusive，独占）	该缓存行中的数据是 **有效** 的，与主内存中的数据 **一致**。并且，这份数据只存在于 **当前核心** 的缓存中。	这是最理想的状态。核心可以直接修改数据，而无需通知其他核心，修改后状态变为 M。
S（Shared，共享）	该缓存行中的数据是 **有效** 的，与主内存中的数据 **一致**。而且，这份数据可能同时存在于 **多个核心** 的缓存中。	当核心需要修改此状态的数据时，必须 **先向所有其他核心广播一个“失效”请求**，要求它们把对应的缓存行标记为无效。收到确认后，当前核心才能修改本地缓存中的值，修改后状态变为 M。
I（Invalid，无效）	该缓存行中的数据是 **无效**的。	要使用它，必须重新从主内存或其他核心的缓存中读取。

MESI 流程



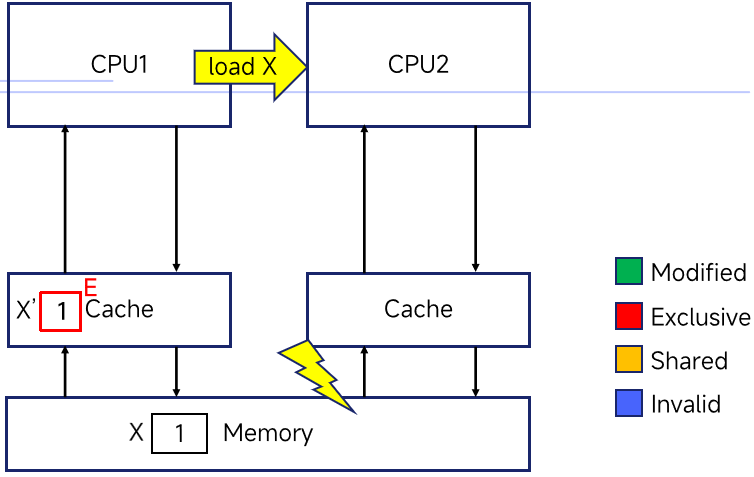
对称多处理
Symmetric Multi-Processing

MESI 流程 – CPU1 读取 X



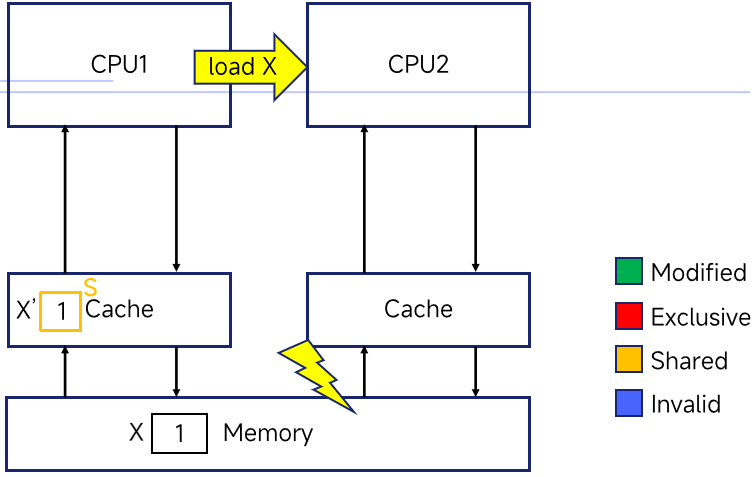
对称多处理
Symmetric Multi-Processing

MESI 流程 – CPU2 读取 X （1）



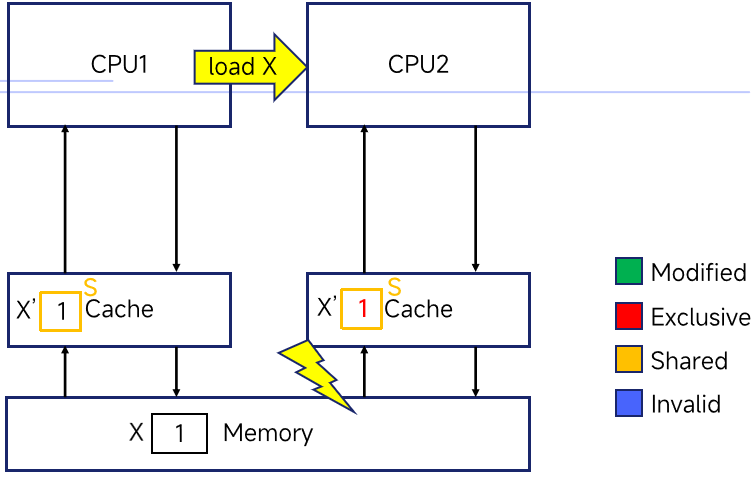
对称多处理
Symmetric Multi-Processing

MESI 流程 – CPU2 读取 X （2）



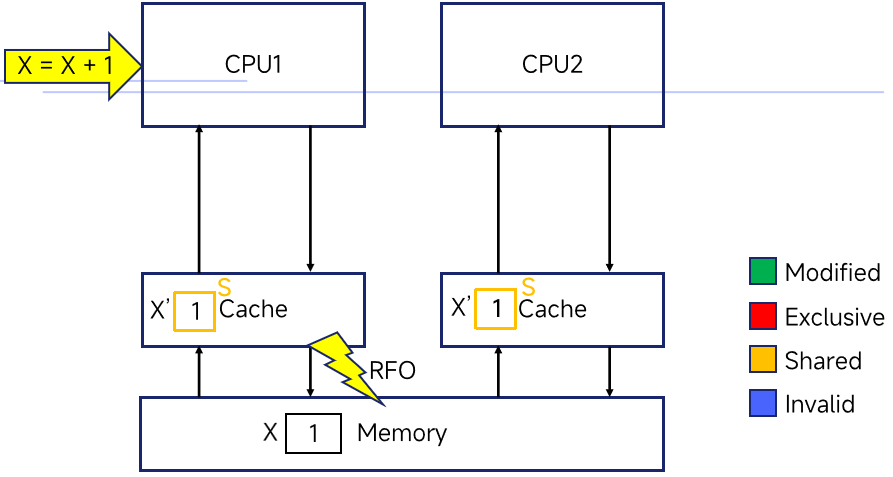
对称多处理
Symmetric Multi-Processing

MESI 流程 – CPU2 读取 X （3）



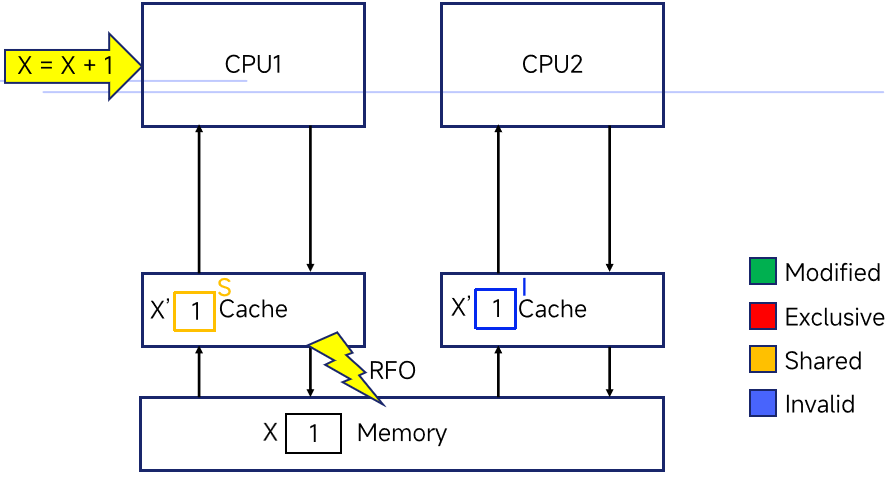
对称多处理
Symmetric Multi-Processing

MESI 流程 – CPU1 执行 X=X+1 （1）



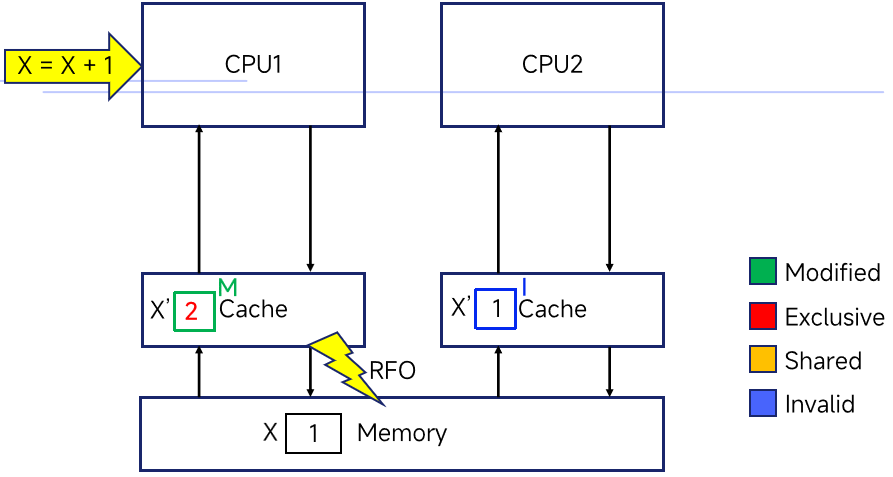
对称多处理
Symmetric Multi-Processing

MESI 流程 – CPU1 执行 X=X+1 （2）



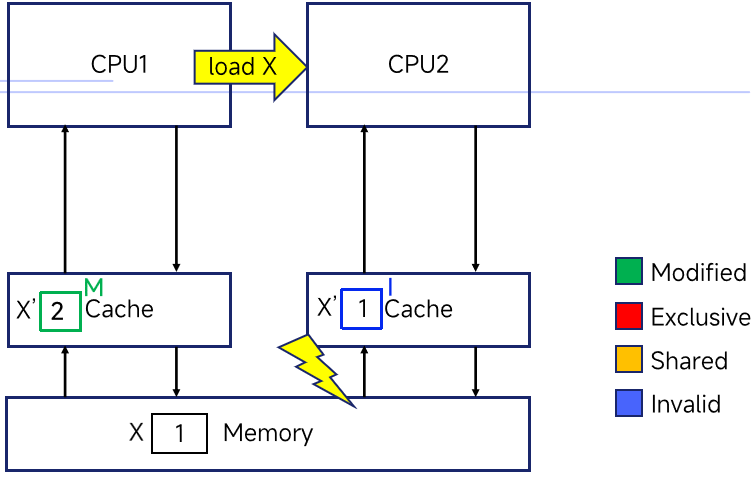
对称多处理
Symmetric Multi-Processing

MESI 流程 – CPU1 执行 X=X+1 （3）



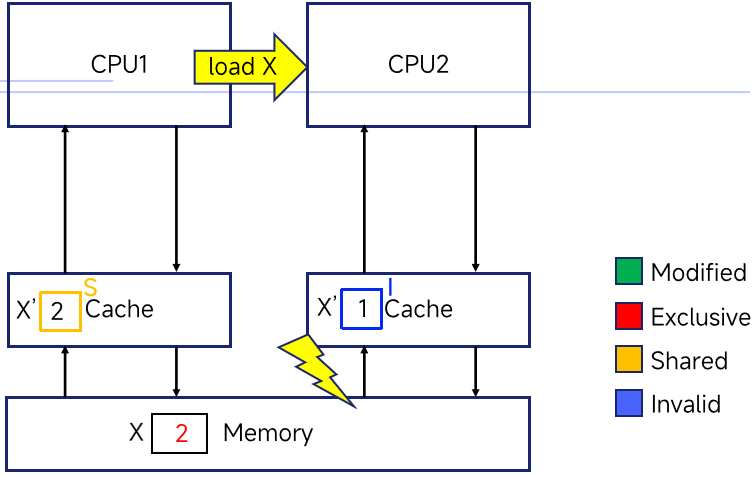
对称多处理
Symmetric Multi-Processing

MESI 流程 – CPU2 再次读取 X （1）



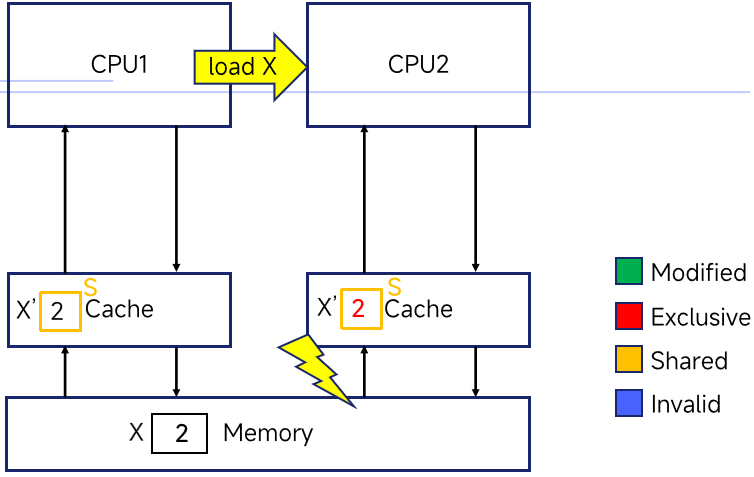
对称多处理
Symmetric Multi-Processing

MESI 流程 – CPU2 再次读取 X （2）



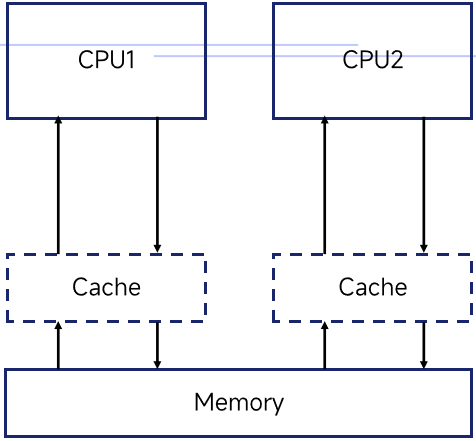
对称多处理
Symmetric Multi-Processing

MESI 流程 – CPU2 再次读取 X （3）

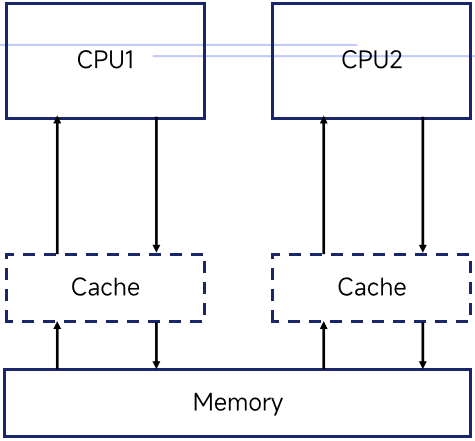


对称多处理
Symmetric Multi-Processing

内存一致性模型 -顺序一致性（Sequential Consistency）

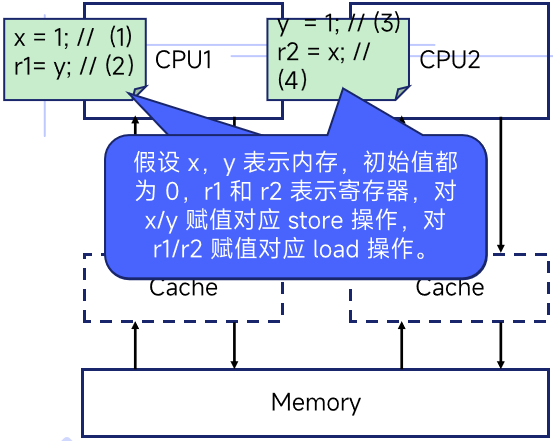


内存一致性模型 -顺序一致性（Sequential Consistency）



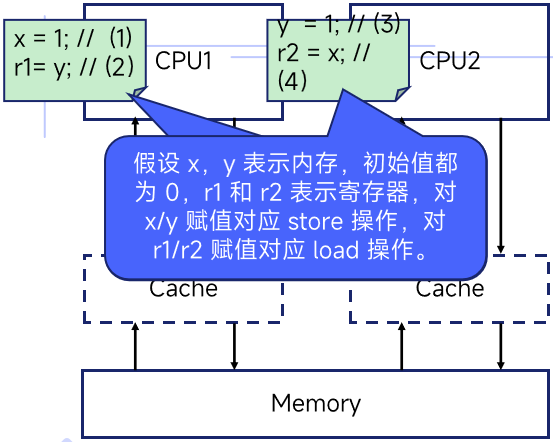
- 条件一：从单个硬件线程的视角来看，每个硬件线程内部的指令都是按照编程的顺序（program order）执行。
- 条件二：任何一个 CPU 对内存的写操作会全局立即生效。

内存一致性模型 -顺序一致性（Sequential Consistency）



- 条件一：从单个硬件线程的视角来看，每个硬件线程内部的指令都是按照编程的顺序（program order）执行。
- 条件二：任何一个 CPU 对内存的写操作会全局立即生效。

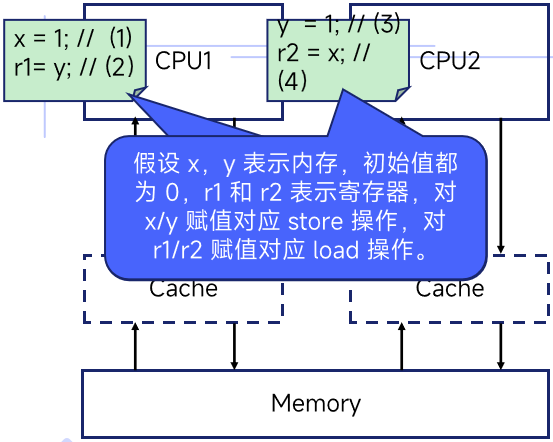
内存一致性模型 -顺序一致性（Sequential Consistency）



- 条件一：从单个硬件线程的视角来看，每个硬件线程内部的指令都是按照编程的顺序（program order）执行。
- 条件二：任何一个 CPU 对内存的写操作会全局立即生效。

流程	结果
(1)->(2)->(3)->(4)	r1 = 0；r2 = 1
(1)->(3)->(2)->(4)	r1 = 1；r2 = 1
(1)->(3)->(4)->(2)	r1 = 1；r2 = 1
(3)->(4)->(1)->(2)	r1 = 1；r2 = 0
(3)->(1)->(2)->(4)	r1 = 1；r2 = 1
(3)->(1)->(4)->(2)	r1 = 1；r2 = 1

内存一致性模型 -顺序一致性（Sequential Consistency）

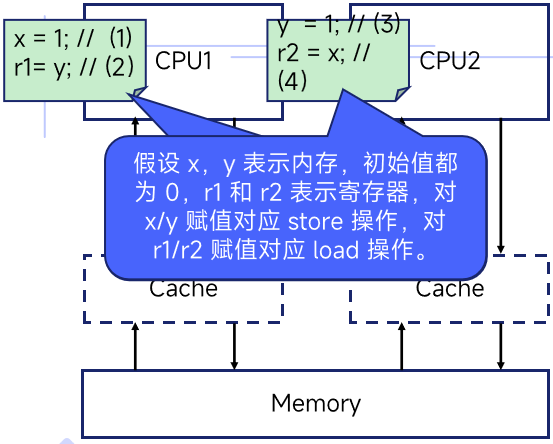


- 条件一：从单个硬件线程的视角来看，每个硬件线程内部的指令都是按照编程的顺序（program order）执行。
- 条件二：任何一个 CPU 对内存的写操作会全局立即生效。

流程	结果
(1)->(2)->(3)->(4)	r1 =
(1)->(3)->(2)->(4)	
(1)->(3)->(4)->(2)	
(3)->(4)->(1)->(2)	r1 0
(3)->(1)->(2)->(4)	r1 = 1； r = 1
(3)->(1)->(4)->(2)	r1 = 1； r2 = 1

在满足条件一的前提下，不可能出现 (2)->(1)->(4)->(3)

内存一致性模型 -顺序一致性（Sequential Consistency）

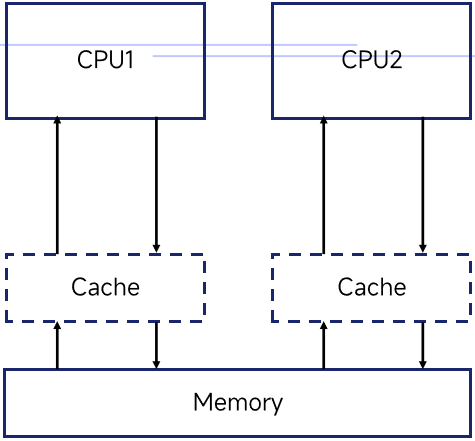


- 条件一：从单个硬件线程的视角来看，每个硬件线程内部的指令都是按照编程的顺序（program order）执行。
- 条件二：任何一个 CPU 对内存的写操作会全局立即生效。

流程	结果
(1)->(2)->(3)->(4)	r1 = 0；r2 = 1
(1)->(3)->(2)->(4)	r1 = 1；r2 = 1
(1)->(3)->(2)->(4)	r1 = 0；r2 = 0
(3)->(4)->(1)->(2)	r1 = 1；r2 = 0
(3)->(1)->(2)->(4)	
(3)->(1)->(4)->(2)	

在满足条件二的前提下，这样的结果不可能出现

内存一致性模型 -顺序一致性（Sequential Consistency）

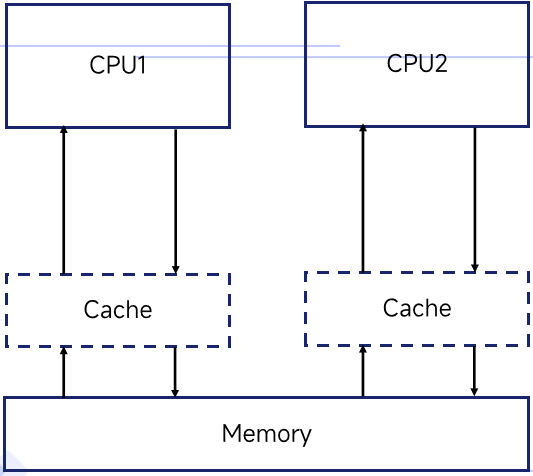


- 条件一：从单个硬件线程的视角来看，每个硬件线程内部的指令都是按照编程的顺序（program order）执行。
- 条件二：任何一个 CPU 对内存的写操作会全局立即生效。

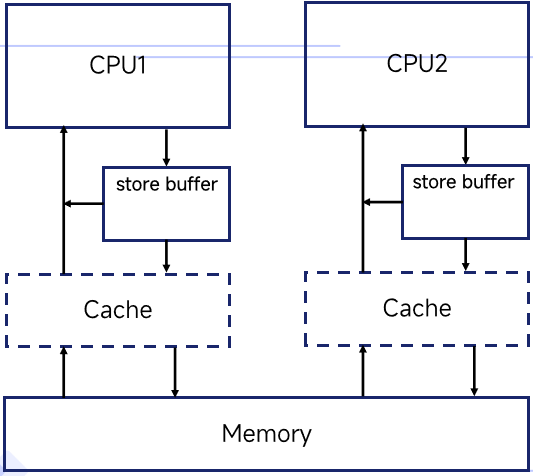
SC 模型是一个标准的理想模型，符合程序员的直觉。

- 条件一使得我们在编写单线程代码时只要按照顺序的逻辑去考虑问题就好了；
- 条件二确保我们在考虑多线程时可以将多线程的操作按逻辑上的全局顺序简单地交错进行；这得益于模型确保任何一个 CPU 对内存的写操作会全局立即生效。

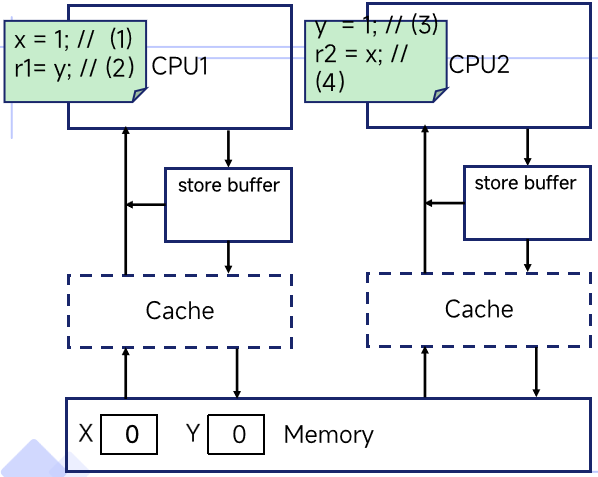
内存一致性模型 -更宽松的内存一致性模型



内存一致性模型 -更宽松的内存一致性模型



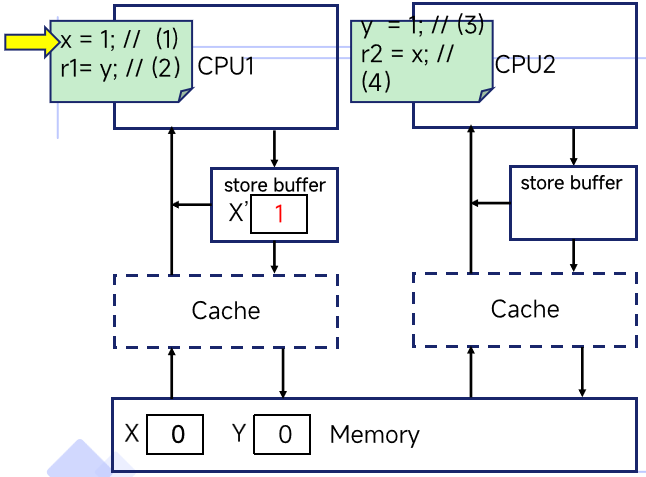
内存一致性模型 -更宽松的内存一致性模型



流程	结果
(1)->(2)->(3)->(4)	r1 = 0； r2 = 1
(1)->(3)->(2)->(4)	r1 = 1； r2 = 1
(1)->(3)->(4)->(2)	r1 = 1； r2 = 1
(3)->(1)->(2)->(4)	r1 = 1； r2 = 0
(3)->(1)->(4)->(2)	r1 = 1； r2 = 1
(3)->(4)->(1)->(2)	r1 = 1； r2 = 1

在 SC 模型下不可能出现 r1 = 0； r2 = 0 的结果

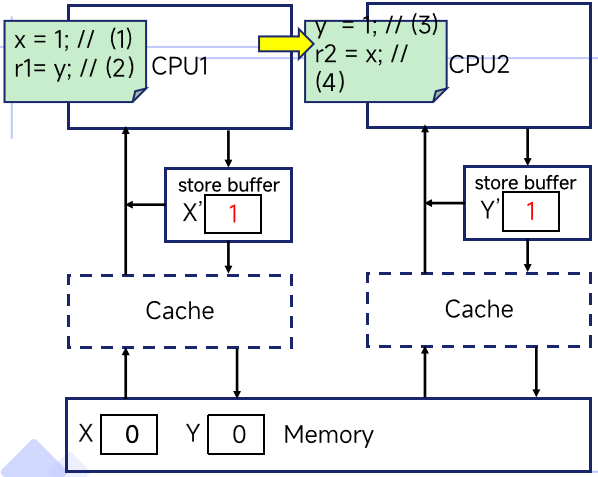
内存一致性模型 -更宽松的内存一致性模型



流程	结果
(1)->(2)->(3)->(4)	r1 = 0； r2 = 1
(1)->(3)->(2)->(4)	r1 = 1； r2 = 1
(1)->(3)->(4)->(2)	r1 = 1； r2 = 1
(3)->(1)->(2)->(4)	r1 = 1； r2 = 0
(3)->(1)->(4)->(2)	r1 = 1； r2 = 1
(3)->(4)->(1)->(2)	r1 = 1； r2 = 1

在 SC 模型下不可能出现 r1 = 0； r2 = 0 的结果

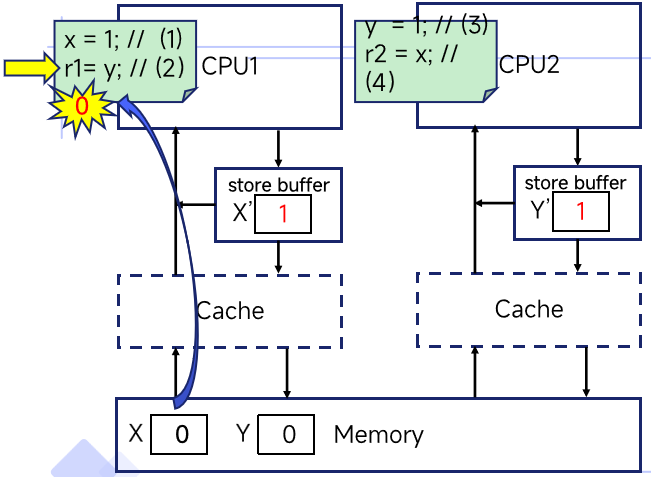
内存一致性模型 -更宽松的内存一致性模型



流程	结果
(1)->(2)->(3)->(4)	r1 = 0； r2 = 1
(1)->(3)->(2)->(4)	r1 = 1； r2 = 1
(1)->(3)->(4)->(2)	r1 = 1； r2 = 1
(3)->	= 0
(3)->	= 1
(3)->	= 1

在 SC 模型下不可能出现 r1 = 0； r2 = 0 的结果

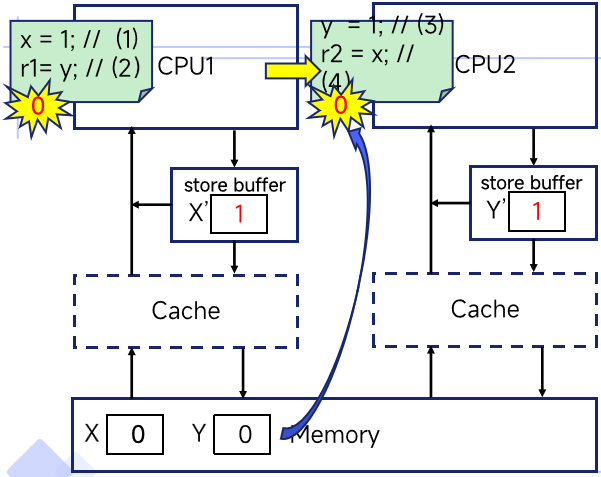
内存一致性模型 -更宽松的内存一致性模型



流程	结果
(1)->(2)->(3)->(4)	r1 = 0； r2 = 1
(1)->(3)->(2)->(4)	r1 = 1； r2 = 1
(1)->(3)->(4)->(2)	r1 = 1； r2 = 1
(3)->(1)->(2)->(4)	r1 = 1； r2 = 0
(3)->(1)->(4)->(2)	r1 = 1； r2 = 1
(3)->(4)->(1)->(2)	r1 = 1； r2 = 1

在 SC 模型下不可能出现 r1 = 0； r2 = 0 的结果

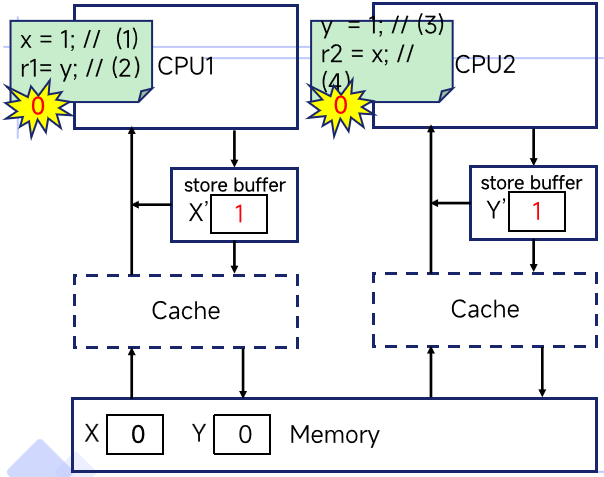
内存一致性模型 -更宽松的内存一致性模型



流程	结果
(1)->(2)->(3)->(4)	r1 = 0； r2 = 1
(1)->(3)->(2)->(4)	r1 = 1； r2 = 1
(1)->(3)->(4)->(2)	r1 = 1； r2 = 1
(3)->(1)->(2)->(4)	r1 = 1； r2 = 0
(3)->(1)->(4)->(2)	r1 = 1； r2 = 1
(3)->(4)->(1)->(2)	r1 = 1； r2 = 1

在 SC 模型下不可能出现 r1 = 0； r2 = 0 的结果

内存一致性模型 -更宽松的内存一致性模型

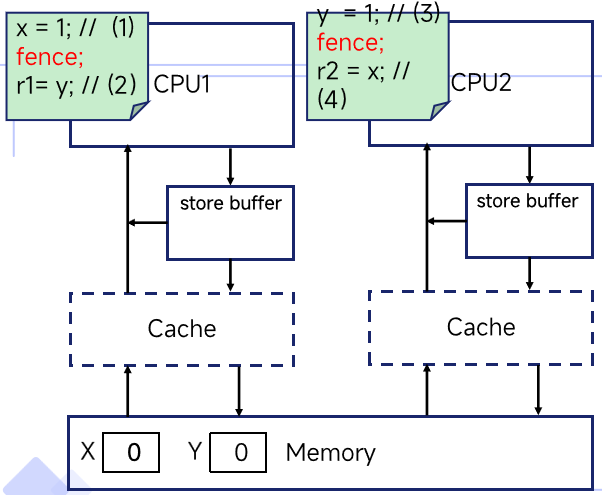


流程	结果
(1)->(2)->(3)->(4)	r1 = 0； r2 = 1
(1)->(3)->(2)->(4)	r1 = 1； r2 = 1
(1)->(3)->(4)->(2)	r1 = 1； r2 = 1
(3)->(1)->(2)->(4)	r2 = 0
(3)->(1)->(4)->(2)	r2 = 1
(3)->(4)->(1)->(2)	r2 = 1

在 SC 模型下不可能出现 r1 = 0； r2 = 0 的结果

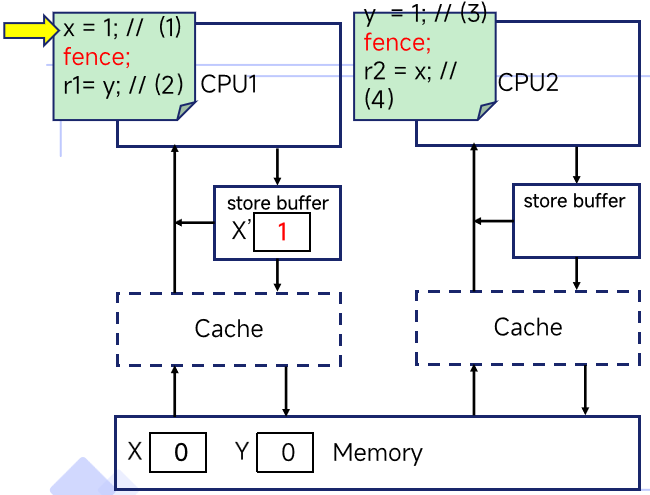
除非出现了乱序?
譬如:
(2)->(4)->(1)->(3)

内存屏障指令



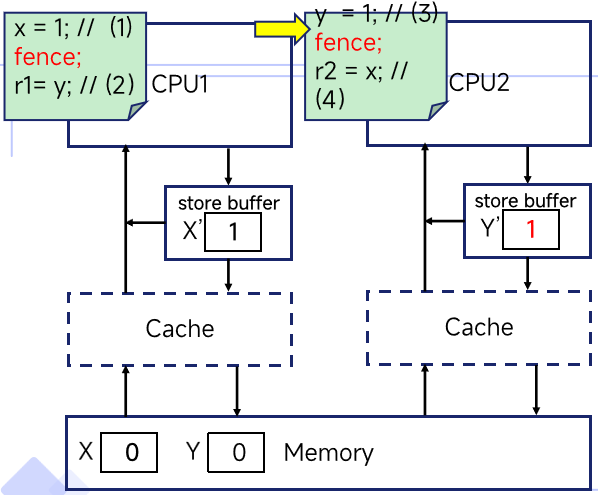
流程	结果
(1)->(2)->(3)->(4)	r1 = 0； r2 = 1
(1)->(3)->(2)->(4)	r1 = 1； r2 = 1
(1)->(3)->(4)->(2)	r1 = 1； r2 = 1
(3)->(4)->(1)->(2)	r1 = 1； r2 = 0
(3)->(1)->(2)->(4)	r1 = 1； r2 = 1
(3)->(1)->(4)->(2)	r1 = 1； r2 = 1

内存屏障指令



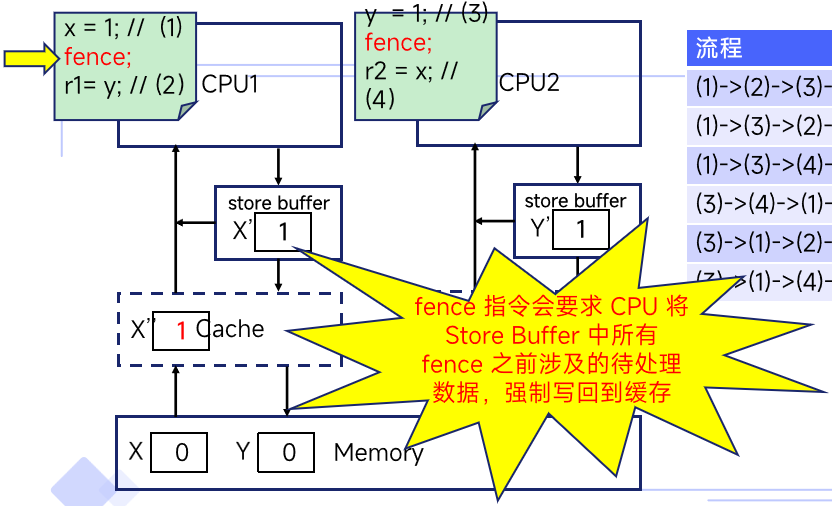
流程	结果
(1)->(2)->(3)->(4)	r1 = 0； r2 = 1
(1)->(3)->(2)->(4)	r1 = 1； r2 = 1
(1)->(3)->(4)->(2)	r1 = 1； r2 = 1
(3)->(4)->(1)->(2)	r1 = 1； r2 = 0
(3)->(1)->(2)->(4)	r1 = 1； r2 = 1
(3)->(1)->(4)->(2)	r1 = 1； r2 = 1

内存屏障指令



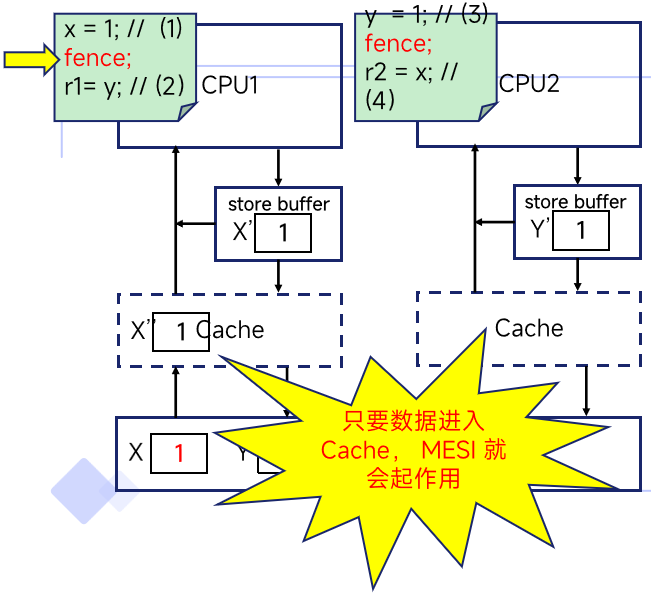
流程	结果
(1)->(2)->(3)->(4)	r1 = 0； r2 = 1
(1)->(3)->(2)->(4)	r1 = 1； r2 = 1
(1)->(3)->(4)->(2)	r1 = 1； r2 = 1
(3)->(4)->(1)->(2)	r1 = 1； r2 = 0
(3)->(1)->(2)->(4)	r1 = 1； r2 = 1
(3)->(1)->(4)->(2)	r1 = 1； r2 = 1

内存屏障指令



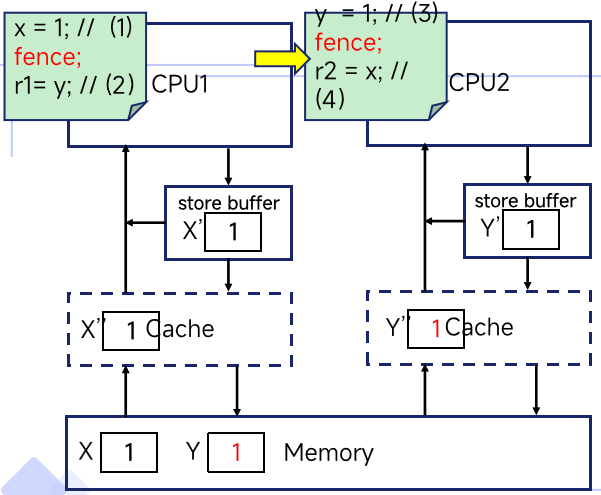
流程	结果
(1)->(2)->(3)->(4)	r1 = 0； r2 = 1
(1)->(3)->(2)->(4)	r1 = 1； r2 = 1
(1)->(3)->(4)->(2)	r1 = 1； r2 = 1
(3)->(4)->(1)->(2)	r1 = 1； r2 = 0
(3)->(1)->(2)->(4)	r1 = 1； r2 = 1
(3)->(1)->(4)->(2)	r1 = 1； r2 = 1

内存屏障指令



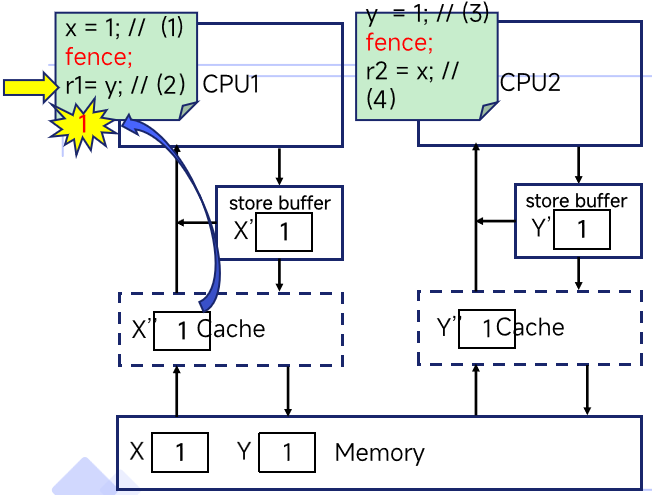
流程	结果
(1)->(2)->(3)->(4)	r1 = 0； r2 = 1
(1)->(3)->(2)->(4)	r1 = 1； r2 = 1
(1)->(3)->(4)->(2)	r1 = 1； r2 = 1
(3)->(4)->(1)->(2)	r1 = 1； r2 = 0
(3)->(1)->(2)->(4)	r1 = 1； r2 = 1
(3)->(1)->(4)->(2)	r1 = 1； r2 = 1

内存屏障指令



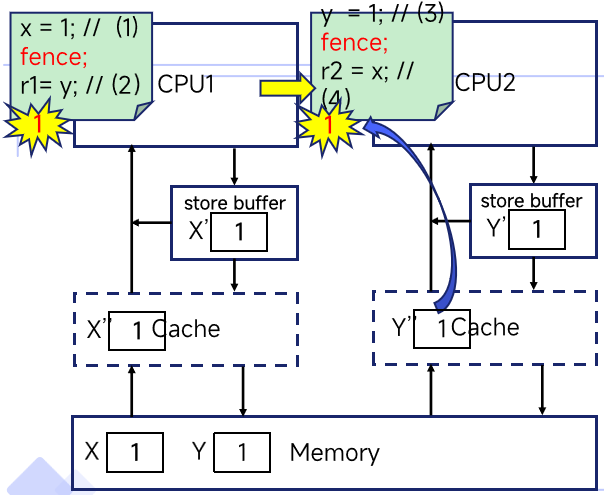
流程	结果
(1)->(2)->(3)->(4)	r1 = 0； r2 = 1
(1)->(3)->(2)->(4)	r1 = 1； r2 = 1
(1)->(3)->(4)->(2)	r1 = 1； r2 = 1
(3)->(4)->(1)->(2)	r1 = 1； r2 = 0
(3)->(1)->(2)->(4)	r1 = 1； r2 = 1
(3)->(1)->(4)->(2)	r1 = 1； r2 = 1

内存屏障指令



流程	结果
(1)->(2)->(3)->(4)	r1 = 0； r2 = 1
(1)->(3)->(2)->(4)	r1 = 1； r2 = 1
(1)->(3)->(4)->(2)	r1 = 1； r2 = 1
(3)->(4)->(1)->(2)	r1 = 1； r2 = 0
(3)->(1)->(2)->(4)	r1 = 1； r2 = 1
(3)->(1)->(4)->(2)	r1 = 1； r2 = 1

内存屏障指令



流程	结果
(1)->(2)->(3)->(4)	r1 = 0； r2 = 1
(1)->(3)->(2)->(4)	r1 = 1； r2 = 1
(1)->(3)->(4)->(2)	r1 = 1； r2 = 1
(3)->(4)->(1)->(2)	r1 = 1； r2 = 0
(3)->(1)->(2)->(4)	r1 = 1； r2 = 1
(3)->(1)->(4)->(2)	r1 = 1； r2 = 1

内存重排序（Memory Ordering）

内存重排序（Memory Ordering）内存操作的执行顺序或可见顺序与程序代码顺序不一致的现象。它可能由编译器优化、CPU 乱序执行、多核系统上 Store Buffer 等引入的缓存一致性问题共同导致。

- **编译器重排序**：在编译阶段（将高级代码转换成指令时），为了优化（如寄存器复用、指令流水线），可能会改变两个无关内存操作的顺序。
- **CPU 重排序**：处理器在执行时，为了提升指令级并行度（如乱序执行），可能会动态调整内存操作的顺序。
- **非理想内存一致性模型（导致的乱序）**：在多核 CPU 的系统中，即使 CPU 是按顺序执行的，由于类似 store buffer 这类缓存的存在，也可能导致其他核心看到的内存操作顺序与代码顺序不同。

内存重排序（Memory Ordering） - 解决方案

针对不同层次的问题原因，采取不同的防止乱序的手段：

- 对于**编译器重排**，我们可以在代码中插入编译器级别的屏障（Compiler barriers），但 Compiler barriers 只对编译器的编译行为有作用，并不会影响硬件的乱序执行和内存一致性问题。
- 对于 **CPU 重排序** 和 **内存一致性模型造成的乱序问题**，我们可以在代码中插入硬件级别的 CPU barrier 指令（如 fence）。

内存重排序（Memory Ordering） - 解决方案

```
void acquire(struct spinlock *lk)
{
  push_off();
  if(holding(lk))
    panic("acquire");
  while(__sync_lock_te
    ;
  __sync_synchronize()
  lk->cpu = mycpu();
}
```

```
void release(struct spinlock *lk)
{
  if(!holding(lk))
    panic("release");
  lk->cpu = 0;
  __sync_synchronize();
  __sync_lock_release(&lk->locked);
  pop_off();
}
```

riscv64-unknown-elf-objdump -S kernel/spinlock.o

```
0000000000000090 <acquire>:
{
  90:   1101                    addi    sp,sp,-32
  // ......
00000000000000b2 <.L16>:
  while(__sync_lock_test_and_set(&lk->locked, 1) != 0)
  b2:   87ba                    mv      a5,a4
  b4:   0cf4a7af                amoswap.w.aq    a5,a5,(s1)
  b8:   2781                    sext.w  a5,a5
  ba:   ffe5                    bnez    a5,b2 <.L16>
  __sync_synchronize();
  bc:   0ff0000f                fence
  lk->cpu = mycpu();
  c0:   00000097                auipc   ra,0x0
  c4:   000080e7                jalr    ra # c0 <.L16+0xe>
  // ......
00000000000000c8 <.LVL19>:
  c8:   e888                    sd      a0,16(s1)
}
```

03

SMP 设计

改动一：运行环境升级为 SMP

```
diff --git a/Makefile b/Makefile
index f5f7310..dda899b 100644
--- a/Makefile
+++ b/Makefile
@@ -95,7 +95,7 @@ QEMUGDB = $(shell if $(QEMU) -help | grep -q '^-gdb'; \
         then echo "-gdb tcp::$(GDBPORT)"; \
         else echo "-s -p $(GDBPORT)"; fi)
 ifndef CPUS
-CPUS := 1
+CPUS := 3
 endif
```

```
diff --git a/kernel/param.h b/kernel/param.h
index 99551a9..2d2801a 100644
--- a/kernel/param.h
+++ b/kernel/param.h
@@ -1,2 +1,2 @@
 #define NPROC        64  // maximum number of processes
-#define NCPU          1  // maximum number of CPUs
+#define NCPU          8  // maximum number of CPUs
```

改动二：区分多核 hartid（1）

```
        # qemu -kernel loads the kernel at 0x80000000
        # and causes each hart (i.e. CPU) to jump there.
        # kernel.ld causes the following code to
        # be placed at 0x80000000.
.section .text
.global _entry
_entry:
        # set up a stack for C.
        # stack0 is declared in start.c,
        # with a 4096-byte stack per CPU.
        # sp = stack0 + ((hartid + 1) * 4096)
        la sp, stack0
        li a0, 1024*4
        csrr a1, mhartid
        addi a1, a1, 1
        mul a0, a0, a1
        add sp, sp, a0
        # jump to start() in start.c
        call start
spin:
        j spin
```

改动二：区分多核 hartid（2）

```
diff --git a/kernel/start.c b/kernel/start.c
index 45fd802..b9391d6 100644
--- a/kernel/start.c
+++ b/kernel/start.c
@@ -40,6 +40,10 @@ start()
   // ask for clock interrupts.
   timerinit();

+  // keep each CPU's hartid in its tp register, for cpuid().
+  int id = r_mhartid();
+  w_tp(id);
+
   // switch to supervisor mode and jump to main().
   asm volatile("mret");
 }
```

改动二：区分多核 hartid（2）

```
diff --git a/kernel/start.c b/kernel/start.c
index 45fd802..b9391d6 100644
--- a/kernel/start.c
+++ b/kernel/start.c
@@ -40,6 +40,10 @@ start()
   // ask for clock interrupts.
   timerinit();

+  // keep each CPU's hartid in
+  int id = r_mhartid();
+  w_tp(id);
+
   // switch to supervisor mode
   asm volatile("mret");
 }
```

```
diff --git a/kernel/proc.c b/kernel/proc.c
index b91e369..2146727 100644
--- a/kernel/proc.c
+++ b/kernel/proc.c
+// Must be called with interrupts disabled,
+// to prevent race with process being moved
+// to a different CPU.
 int
 cpuid()
 {
-  return 0;
+  int id = r_tp();
+  return id;
 }
```

改动二：区分多核 hartid（2）

```
diff --git a/kernel/start.c b/kernel/start.c
index 45fd802..b9391d6 100644
--- a/kernel/start.c
+++ b/kernel/start.c
@@ -40,6 +40,10 @@ start()
   // ask for clock interrupts.
   timerinit();

+  // keep each CPU's hartid in
+  int id = r_mhartid();
+  w_tp(id);
+
   // switch to supervisor mode
   asm volatile("mret");
 }
```

```
diff --git a/kernel/proc.c b/kernel/proc.c
index b91e369..2146727 100644
--- a/kernel/proc.c
+++ b/kernel/p
+// Must be ca
+// to prevent
+// to a diffe
 int
 cpuid()
 {
-  return 0;
+  int id = r_
+  return id;
 }
```

```
diff --git a/kernel/kernelvec.S b/kernel/kernelvec.S
index 324fc18..5561b89 100644
--- a/kernel/kernelvec.S
+++ b/kernel/kernelvec.S
@@ -41,7 +41,7 @@ kernelvec:
         ld ra, 0(sp)
         # ld sp, 8(sp)
         ld gp, 16(sp)
-        ld tp, 24(sp)
+        # not tp (contains hartid), in case we moved CPUs
         ld t0, 32(sp)
         ld t1, 40(sp)
         ld t2, 48(sp)
```

改动三：多核的启动

```c
void main()
{
  uartinit();
  printfinit();
  printf("\n");
  printf("xv6 kernel is booting\n");
  printf("\n");
  kinit();         // physical page allocator
  procinit();      // process table
  trapinit();      // trap vectors
  trapinithart();  // install kernel trap vector
  plicinit();      // set up interrupt controller
  plicinithart();  // ask PLIC for device interrupts
  userinit();      // first user process

  scheduler();
}
```

```c
volatile static int started = 0;
void main()
{
  if(cpuid() == 0){
    uartinit();
    printfinit();
    printf("\n");
    printf("xv6 kernel is booting\n");
    printf("\n");
    kinit();         // physical page allocator
    procinit();      // process table
    trapinit();      // trap vectors
    trapinithart();  // install kernel trap vector
    plicinit();      // set up interrupt controller
    plicinithart();  // ask PLIC for device interrupts
    userinit();      // first user process
    __sync_synchronize();
    started = 1;
  } else {
    while(started == 0)
      ;
    __sync_synchronize();
    printf("hart %d starting\n", cpuid());
    trapinithart();   // install kernel trap vector
    plicinithart();   // ask PLIC for device interrupts
  }

  scheduler();
}
```

改动四：任务调度对多核的支持 - myproc

```
// Return the current struct proc *, or zero if none.
struct proc*
myproc(void)
{

  struct cpu *c = mycpu();
  struct proc *p = c->proc;

  return p;
}
```

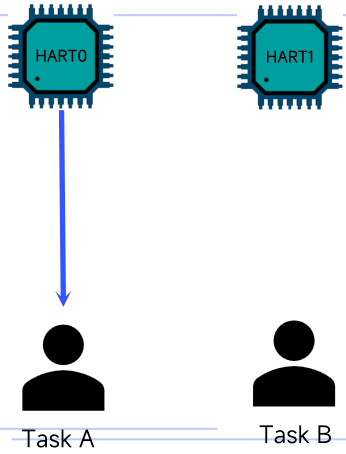
myproc() 在单核上没问题，但在多核上并发情况下会有问题

```
struct cpu {
  struct proc *proc;
  // ......
};
```

改动四：任务调度对多核的支持 - myproc

```c
// Return the current struct proc *, or zero if none.
struct proc*
myproc(void)
{

  struct cpu *c = mycpu();
  struct proc *p = c->proc;

  return p;
}
```

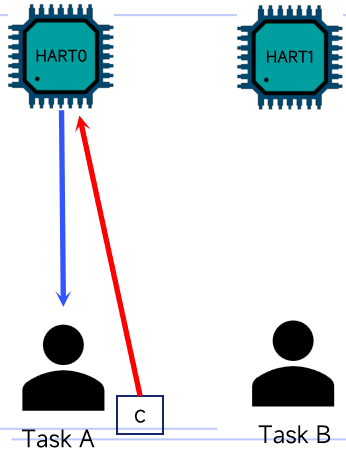
```c
struct cpu {
  struct proc *proc;
  // ......
};
```



改动四：任务调度对多核的支持 - myproc

```c
// Return the current struct proc *, or zero if none.
struct proc*
myproc(void)
{

  struct cpu *c = mycpu();
  struct proc *p = c->proc;

  return p;
}
```

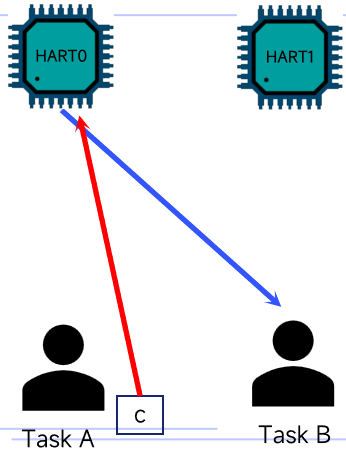
```c
struct cpu {
  struct proc *proc;
  // ......
};
```



改动四：任务调度对多核的支持 - myproc

```c
// Return the current struct proc *, or zero if none.
struct proc*
myproc(void)
{

  struct cpu *c = mycpu();
  struct proc *p = c->proc;

  return p;
}
```

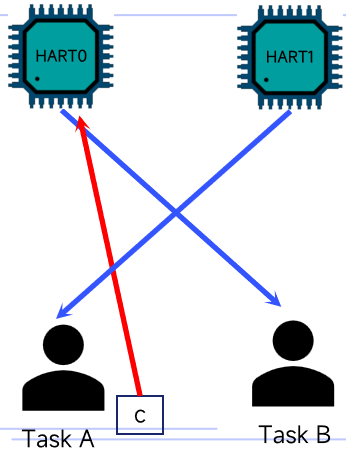
```c
struct cpu {
  struct proc *proc;
  // ......
};
```



改动四：任务调度对多核的支持 - myproc

```c
// Return the current struct proc *, or zero if none.
struct proc*
myproc(void)
{

  struct cpu *c = mycpu();
  struct proc *p = c->proc;

  return p;
}
```

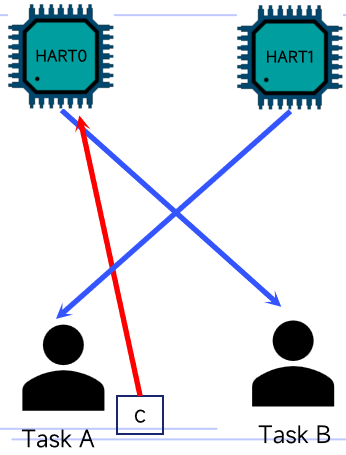
```c
struct cpu {
  struct proc *proc;
  // ......
};
```



改动四：任务调度对多核的支持 - myproc

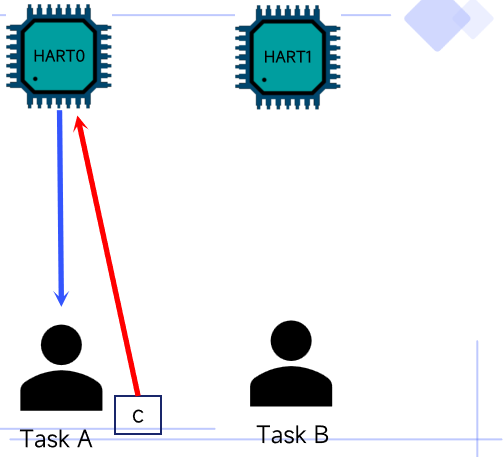
```c
// Return the current struct proc *, or zero if none.
struct proc*
myproc(void)
{

  struct cpu *c = mycpu();
  struct proc *p = c->proc;

  return p;
}
```

```c
struct cpu {
  struct proc *proc;
  // ......
};
```



改动四：任务调度对多核的支持 - myproc

```
diff --git a/kernel/proc.c b/kernel/proc.c
index 5f8c2df..7a646a1 100644
--- a/kernel/proc.c
+++ b/kernel/proc.c
@@ -47,8 +53,10 @@ mycpu(void)
 struct proc*
 myproc(void)
 {
+  push_off();
   struct cpu *c = mycpu();
   struct proc *p = c->proc;
+  pop_off();
   return p;
 }
```



改动五：任务调度对多核的支持 - 任务切换

```
// Per-process state
struct proc {


  enum procstate state;        // Process state
  int pid;                     // Process ID


  uint64 kstack;               // Physical address of kernel stack
  struct context context;      // swtch() here to run process
  void (*start)(void);         // Function to run when process starts
};
```

创建任务或者执行任务切换时会修改任务的 state 和 PID，如果这些工作在多个 CPU 核心上并发执行时会引入竞争，需要上锁实现同步。

改动五：任务调度对多核的支持 - 任务切换

```
// Per-process state
struct proc {
  struct spinlock lock;

  // p->lock must be held when using these:
  enum procstate state;        // Process state
  int pid;                     // Process ID

  // these are private to the process, so p->lock need not be held.
  uint64 kstack;               // Physical address of kernel stack
  struct context context;      // swtch() here to run process
  void (*start)(void);         // Function to run when process starts
};
```

采用了每个进程一个锁的细粒度设计。

- 减少锁竞争 - 不同进程的操作可以并行
- 提高并发性 - 锁的粒度更细

改动五：任务调度对多核的支持 - 任务切换

```
diff --git a/kernel/proc.c b/kernel/proc.c
index 5f8c2df..7a646a1 100644
--- a/kernel/proc.c
+++ b/kernel/proc.c
 static struct proc*
 allocproc(void (*start_routin)(void))
 {
   struct proc *p;

   for(p = proc; p < &proc[NPROC]; p++) {
+    acquire(&p->lock);
     if(p->state == UNUSED) {
       goto found;
+    } else {
+      release(&p->lock);
     }
   }
   return 0;
```

```
// Set up first user process.
void
userinit(void)
{
  struct proc *p;

  p = allocproc(user_task0);
  if (p == 0)
    panic("userinit: allocproc task0 failed");
  p->state = RUNNABLE;
+ release(&p->lock);

  p = allocproc(user_task1);
  if (p == 0)
    panic("userinit: allocproc task1 failed");
  p->state = RUNNABLE;
+ release(&p->lock);
}
```

改动五：任务调度对多核的支持 - 任务切换

```
void scheduler(void)
{
  struct proc *p;
  struct cpu *c = mycpu();

  c->proc = 0;
  for(;;){
    for(p = proc; p < &proc[NPROC]; p++) {
+     acquire(&p->lock);
      if(p->state == RUNNABLE) {
-       // Switch to chosen process.
+       // Switch to chosen process.  It is the process's job
+       // to release its lock and then reacquire it
+       // before jumping back to us.
        p->state = RUNNING;
        c->proc = p;
        swtch(&c->context, &p->context);

        // Process is done running for now.
        // It should have changed its p->state before coming back.
        c->proc = 0;
      }
+     release(&p->lock);
    }
  }
}
```

```
// Give up the CPU for one scheduling round.
void
yield(void)
{
  struct proc *p = myproc();
+ acquire(&p->lock);
  p->state = RUNNABLE;
  sched();
+ release(&p->lock);
}
```

```
void sched(void)
{
  int intena;
  struct proc *p = myproc();

+ if(!holding(&p->lock))
+   panic("sched p->lock");
+ if(mycpu()->noff != 1)
+   panic("sched locks");
  if(p->state == RUNNING)
    panic("sched RUNNING");
  if(intr_get())
    panic("sched interruptible");

  intena = mycpu()->intena;
  swtch(&p->context, &mycpu()->context);
  mycpu()->intena = intena;
}
```

演示 SMP 上的多任务运行

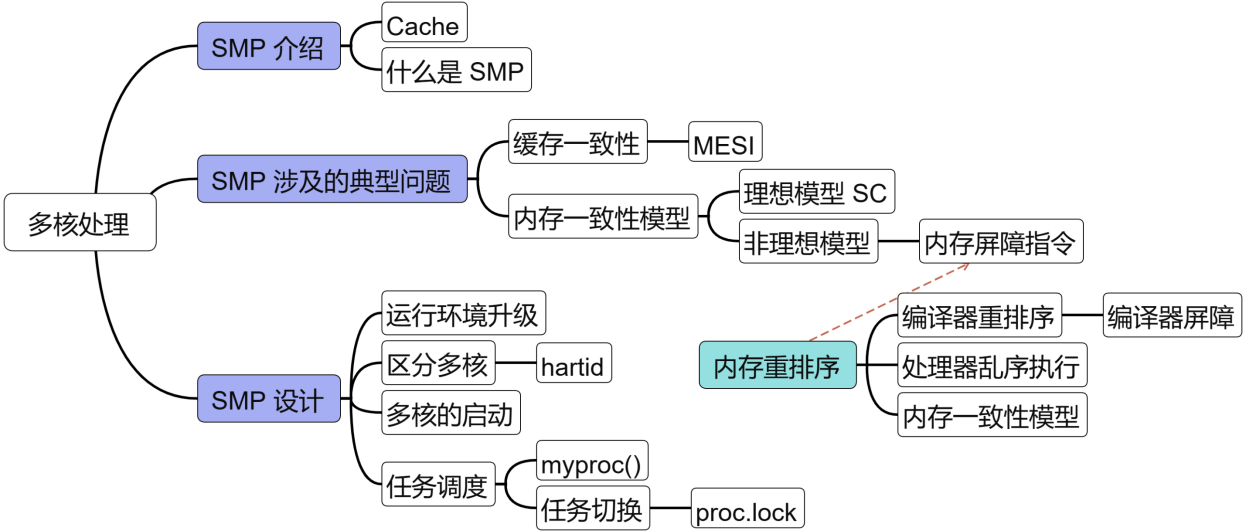
```c
void user_task0(void)
{
  printf("Task 0: Created! cpu[%d]\n", cpuid());
  while (1) {
    printf("Task 0: Running... cpu[%d]\n", cpuid());
    task_delay(DELAY);
  }
}

void user_task1(void)
{
  printf("Task 1: Created! cpu[%d]\n", cpuid());
  while (1) {
    printf("Task 1: Running... cpu[%d]\n", cpuid());
    task_delay(DELAY);
  }
}
```

演示 SMP 上的多任务运行

```c
void user_task0(void)
{
  printf("Task 0: Created! cpu[%d]\n", cp
  while (1) {
    printf("Task 0: Running... cpu[%d]\n"
    task_delay(DELAY);
  }
}

void user_task1(void)
{
  printf("Task 1: Created! cpu[%d]\n", cp
  while (1) {
    printf("Task 1: Running... cpu[%d]\n"
    task_delay(DELAY);
  }
}
```

```
qemu-system-riscv64 -machine virt -bios none -kernel kerne
l -m 128M -smp 3 -nographic

xv6 kernel is booting

hart 1 starting
hart 2 starting
Task 1: Created! cpu[0]
Task 1: Running... cpu[0]
Task 0: Created! cpu[1]
Task 0: Running... cpu[1]
Task 1: Running... cpu[1]
Task 0: Running... cpu[0]
Task 0: Running... cpu[1]
Task 1: Running... cpu[0]
Task 0: Running... cpu[2]
Task 1: Running... cpu[1]
Task 0: Running... cpu[2]
Task 1: Running... cpu[0]
Task 0: Running... cpu[1]
Task 1: Running... cpu[0]
Task 0: Running... cpu[0]
Task 1: Running... cpu[1]
Task 0: Running... cpu[1]
Task 1: Running... cpu[2]
Task 0: Running... cpu[2]
Task 1: Running... cpu[1]
Task 0: Running... cpu[2]
```

本章总结


多核处理
SMP 介绍
Cache
什么是 SMP
SMP 涉及的典型问题
缓存一致性
MESI
内存一致性模型
理想模型 SC
非理想模型
内存屏障指令
SMP 设计
运行环境升级
区分多核
hartid
多核的启动
任务调度
myproc()
任务切换
proc.lock
内存重排序
编译器重排序
编译器屏障
处理器乱序执行
内存一致性模型

谢谢